Interception de Trafic Mobile

avec Mobexler & Burp Suite

Auteur :	BOUSEDRA Ilyas
Date :	February 21, 2026
Environnement :	Mobexler VM
Outil principal :	Burp Suite Community
Cible :	Émulateur Android (labo)
Classification :	Usage pédagogique

"Comprendre le trafic mobile avant de le sécuriser."

Ce rapport documente les étapes d'un laboratoire d'analyse de trafic réseau
dans un cadre pédagogique contrôlé. Toute utilisation hors cadre autorisé est prohibée.

Contents

1 Présentation du laboratoire **2**
1.1 Objectif général . . . 2
1.2 Environnement Mobexler . . . 2
1.3 Schéma d'architecture . . . 2

2 Étape 1 — Lancement de Burp Suite **3**
2.1 Description . . . 3
2.2 Commande de lancement . . . 3
2.3 Procédure . . . 3

3 Étape 2 — Vérification du Proxy Listener **4**
3.1 Description . . . 4
3.2 Procédure . . . 4
3.3 Configuration recommandée . . . 4

4 Étape 3 — Identification de l'adresse IP hôte **6**
4.1 Description . . . 6
4.2 Commandes . . . 6
4.3 Interprétation des interfaces réseau . . . 6

5 Étape 4 — Configuration du proxy côté émulateur Android **8**
5.1 Description . . . 8
5.2 Procédure sur l'émulateur . . . 8

6 Étape 5 — Premier test : capture HTTP **10**
6.1 Description . . . 10
6.2 Procédure . . . 10
6.3 Diagnostic si aucun trafic n'apparaît . . . 10

7 Étape 6 — Lecture analytique d'une requête **12**
7.1 Description . . . 12
7.2 Procédure . . . 12
7.3 Tableau d'analyse des éléments . . . 13

8 Étape 7 — Interception contrôlée (mode pédagogique) **14**
8.1 Description . . . 14
8.2 Procédure . . . 14

9 Étape 8 — Gestion du certificat CA pour HTTPS **16**
9.1 Description . . . 16
9.2 Export du certificat Burp . . . 16
9.3 Installation dans Android . . . 16

10 Étape 9 — Production du mini-rapport **18**
10.1 Description 18
10.2 Structure du rapport 18
10.3 Contenu requis du rapport 18

11 Nettoyage — Hygiène de fin de séance **20**
11.1 Description 20

12 Checkpoints de validation **21**

13 Conclusion **21**

1. Présentation du laboratoire

1.1. Objectif général

Ce laboratoire vise à mettre en place une chaîne complète d'interception et d'analyse du trafic réseau généré par un émulateur Android, en utilisant **Mobexler** comme environnement de travail et **Burp Suite** comme proxy d'interception.

L'objectif pédagogique n'est **pas** de modifier les requêtes ni d'attaquer une cible, mais de :

> Comprendre comment fonctionne un proxy man-in-the-middle en labo,

> Lire et analyser les requêtes HTTP/HTTPS telles qu'elles transitent,

> Produire une preuve documentée (rapport), reproductible.

1.2. Environnement Mobexler

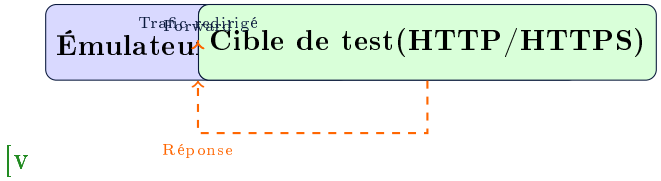
Mobexler est une machine virtuelle (VM) dédiée aux tests de sécurité mobile. Elle embarque nativement :

]Burp Suite Community Edition Un émulateur Android (Genymotion ou AVD) Les outils ADB (Android Debug Bridge) Les outils réseau (nmap, Wireshark, etc.)

[!] Point de vigilance

Tout ce laboratoire se déroule **au sein de la VM Mobexler**. L'émulateur Android et Burp Suite tournent sur la même machine virtuelle, ce qui simplifie la configuration réseau.

1.3. Schéma d'architecture



[v

2. Étape 1 — Lancement de Burp Suite

2.1. Description

La première action consiste à démarrer Burp Suite depuis Mobexler et à créer un projet temporaire de laboratoire. L'interception doit rester **désactivée** au départ.

2.2. Commande de lancement

Commande(s)

```
# Depuis le terminal de Mobexler :
burpsuite &

# Ou depuis le lanceur graphique :
# Applications > Pentest > Burp Suite
```

2.3. Procédure

1. Au démarrage, sélectionner **"Temporary project"**.
2. Cliquer sur **"Use Burp defaults"**.
3. Cliquer sur **"Start Burp"**.
4. Naviguer vers l'onglet **Proxy** → **Intercept**.
5. Vérifier que le bouton affiche **"Intercept is off"** — ne pas l'activer pour l'instant.

[i] Pourquoi cette étape ?

Un proxy d'observation **ne doit pas bloquer le trafic** tant que la configuration n'est pas validée. Démarrer avec l'intercept activé bloquerait toute navigation et rendrait le diagnostic impossible.

[v] À observer / Résultat attendu

L'onglet **HTTP history** est visible.

Le bouton d'interception est accessible mais **désactivé**.

L'interface Burp répond normalement.

[x] Erreurs fréquentes

Démarrer avec l'interception activée → bloque toute navigation.

Se perdre entre les onglets Proxy / Target / Repeater sans vérifier la capture d'abord.

3. Étape 2 — Vérification du Proxy Listener

3.1. Description

Burp Suite écoute sur un port spécifique pour intercepter le trafic. Il faut vérifier que ce listener est actif et configuré pour être accessible depuis l'émulateur.

3.2. Procédure

1. Dans Burp, aller dans **Proxy** → **Proxy settings** (ou **Options** selon la version).
2. Repérer la section **"Proxy Listeners"**.
3. Vérifier que la case **"Running"** est cochée.
4. Vérifier le port (généralement **8080** dans Mobexler).
5. Si l'adresse est `127.0.0.1`, cliquer sur **Edit** et changer en **"All interfaces"** (`0.0.0.0`).

3.3. Configuration recommandée

Commande(s)

```
# V rification depuis le terminal que le port est ouvert :
ss -tlnp | grep 8080

# R sultat attendu :
LISTEN  0  50  0.0.0.0:8080  0.0.0.0:*  users:(("java",pid=XXXX
    ,...))
```

Paramètre	Valeur
Interface d'écoute	`0.0.0.0` (All interfaces)
Port	`8080`
État	Running (activé)

[i] Pourquoi cette étape ?

Le listener sur `0.0.0.0` est nécessaire car l'émulateur Android, même dans Mobexler, peut ne pas voir `127.0.0.1` comme l'hôte local. En mettant **All interfaces**, on s'assure que le trafic provenant de l'émulateur est bien capturé.

[x] Erreurs fréquentes

Listener désactivé (case "Running" décochée).

Listener limité à `127.0.0.1` → l'émulateur ne peut pas joindre le proxy.

Port 8080 déjà utilisé par un autre service → changer pour 8081 ou 9090.

4. Étape 3 — Identification de l'adresse IP hôte

4.1. Description

Pour que l'émulateur puisse envoyer son trafic vers Burp, il doit connaître l'adresse IP de la machine hôte (ici, la VM Mobexler elle-même).

4.2. Commandes

Commande(s)

```
# M thode 1     Afficher toutes les interfaces r seau :
ip a

# M thode 2     Filtrer directement l'IPv4 de eth0 :
ip addr show eth0 | grep "inet " | awk '{print $2}'

# M thode 3     Alternative rapide :
hostname -I

# Exemple de sortie attendue :
192.168.56.101
```

4.3. Interprétation des interfaces réseau

Interface	Type	Usage
`lo`	Loopback	`127.0.0.1` — usage interne uniquement
`eth0` / `ens33`	Réseau principal	IP locale du labo — **à utiliser**
`tun0`	VPN	À éviter (mauvais réseau)
`docker0`	Docker	IP virtuelle containers

[!] Point de vigilance

Dans Mobexler, l'interface réseau à utiliser est souvent **eth0** ou **ens33**. Ne pas utiliser une IP de VPN (`tun0`) ou de container Docker. L'IP doit être dans le même réseau que l'émulateur.

[v] À observer / Résultat attendu

Une adresse IPv4 du type `A.B.C.D` est identifiée.

Cette adresse sera notée `<IP_HOTE>` dans les étapes suivantes.

[x] Erreurs fréquentes

Utiliser une IP VPN active (réseau différent de l'émulateur).

Confondre l'IP publique (WAN) avec l'IP locale (LAN).

Utiliser `127.0.0.1` → inaccessible depuis l'émulateur.

5. Étape 4 — Configuration du proxy côté émulateur Android

5.1. Description

L'émulateur Android doit être configuré pour envoyer son trafic vers Burp Suite. Cela se fait en configurant un proxy manuel dans les paramètres Wi-Fi.

5.2. Procédure sur l'émulateur

1. Ouvrir l'émulateur Android depuis Mobexler (Genymotion ou AVD).
2. Aller dans **Paramètres** → **Wi-Fi**.
3. Faire un **appui long** sur le réseau Wi-Fi connecté.
4. Sélectionner **"Modifier le réseau"** ou **"Modify network"**.
5. Activer les **options avancées**.
6. Dans le champ **Proxy**, sélectionner **"Manuel"**.
7. Renseigner les champs :

Commande(s)

```
# Param tres     saisir dans l'  mulateur   Android :
Proxy hostname : <IP_HOTE>     # Ex: 192.168.56.101
Proxy port     : 8080

# Via ADB (m thode alternative depuis le terminal Mobexler) :
adb shell settings put global http_proxy <IP_HOTE>:8080

# V rification ADB :
adb shell settings get global http_proxy
# R sultat attendu : 192.168.56.101:8080
```

8. Appuyer sur **Enregistrer**.

[i] Pourquoi cette étape ?

Sans cette configuration, le trafic du navigateur Android part directement vers Internet et **n'apparaît pas dans Burp**. Le proxy manuel force tout le trafic HTTP/HTTPS à transiter via Burp Suite.

[v] À observer / Résultat attendu

Le proxy est affiché en mode **"Manuel"** dans les paramètres Wi-Fi.

L'hôte et le port correspondent exactement à ceux des étapes 2 et 3.

La navigation dans l'émulateur génère des entrées dans Burp.

[x] Erreurs fréquentes

Mauvais port saisi (ex. 8008 au lieu de 8080).

Hostname vide ou incorrect.

Proxy configuré sur un réseau Wi-Fi différent de celui actif.

Oublier d'enregistrer après modification.

6. Étape 5 — Premier test : capture HTTP

6.1. Description

Cette étape valide que la chaîne complète (émulateur → proxy → Burp) fonctionne correctement, en utilisant du trafic HTTP simple (non chiffré).

6.2. Procédure

1. Dans l'émulateur, ouvrir le **navigateur** (Chrome ou navigateur natif).
2. Naviguer vers une cible de test autorisée :

Commande(s)

```
# Cibles de test autoris es (exemples) :
http://testphp.vulnweb.com      # Site de test Acunetix
http://natas.labs.overthewire.org  # Cible CTF
http://10.x.x.x                 # Cible interne au r seau de labo

# V rification que la cible est joignable depuis Mobexler :
curl -I http://testphp.vulnweb.com
```

3. Revenir dans Burp Suite.
4. Aller dans **Proxy** → **HTTP history**.
5. Vérifier qu'au moins une requête apparaît dans la liste.

[i] Pourquoi cette étape ?

Cette étape confirme que le routage via proxy fonctionne **avant d'aborder HTTPS**. Si aucun trafic n'apparaît ici, il est inutile de passer à la suite.

6.3. Diagnostic si aucun trafic n'apparaît

Commande(s)

```
# V rifier que le listener Burp   coute   :
ss -tlnp | grep 8080

# V rifier la connectivit    mulateur       h te :
adb shell ping -c 3 <IP_HOTE>

# V rifier le proxy Android configur  :
adb shell settings get global http_proxy

# Tester manuellement via curl depuis l'  mulateur   (ADB shell) :
adb shell curl -x <IP_HOTE>:8080 http://testphp.vulnweb.com
```

[v] À observer / Résultat attendu

Au moins une ligne apparaît dans **HTTP history**.

La méthode (GET), l'URL et le statut (200) sont visibles.

La taille de la réponse est non nulle.

[x] Erreurs fréquentes

Aucun trafic → revenir aux étapes 2, 3, 4.

Interception activée → requêtes bloquées et navigation impossible.

Cible HTTPS testée avant l'installation du certificat.

7. Étape 6 — Lecture analytique d'une requête

7.1. Description

L'analyse d'une requête capturée est la compétence centrale de ce laboratoire. L'objectif est de **lire sans modifier**, en comprenant chaque composant.

7.2. Procédure

1. Dans **HTTP history**, cliquer sur une requête pour la sélectionner.
2. Observer l'onglet **Raw** dans le panneau inférieur.
3. Identifier les éléments suivants :

Commande(s)

```
# Exemple de requ te HTTP brute captur e dans Burp :

GET /listproducts.php?cat=1 HTTP/1.1
Host: testphp.vulnweb.com
User-Agent: Mozilla/5.0 (Linux; Android 11; sdk_gphone_x86)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9
Accept-Language: fr-FR,fr;q=0.9
Accept-Encoding: gzip, deflate
Cookie: PHPSESSID=abc123def456
Connection: keep-alive

#    lments       identifier :
# - M thode       : GET
# - Chemin        : /listproducts.php
# - Param tre     : cat=1  (param tre GET, potentiellement
   injectable)
# - User-Agent    : r  v  le l'OS et le navigateur
# - Cookie        : PHPSESSID (cookie de session)
```

4. Ouvrir le panneau **Inspector** (Burp Suite Pro / récentes versions) pour une vue structurée.

7.3. Tableau d'analyse des éléments

Élément	Ce qu'il révèle	Risque potentiel
Méthode GET	Paramètres en clair dans l'URL	Journalisés côté serveur / proxy
Cookie sans `Secure`	Transit possible en HTTP	Vol de session si HTTP
Cookie sans `HttpOnly`	Accessible via JavaScript	Vulnérabilité XSS
Token dans l'URL	Ex: `?token=xyz`	Exposé dans les logs
`User-Agent`	Version OS/navigateur	Fingerprinting

[i] Pourquoi cette étape ?

La compétence clé en sécurité mobile est **l'analyse** : comprendre ce qui est envoyé, quand, et pourquoi. Un cookie sans attribut `Secure` en HTTP est une observation factuelle — pas une vulnérabilité confirmée sans contexte.

[x] Erreurs fréquentes

Regarder uniquement le corps (body) et ignorer les headers.

Qualifier un cookie de "secret" sans analyser son contexte.

Tirer des conclusions sans horodatage ni version de l'application.

8. Étape 7 — Interception contrôlée (mode pédagogique)

8.1. Description

Cette étape démontre la différence entre le mode **passif** (HTTP history) et le mode **actif** (interception bloquante). L'objectif est uniquement l'observation, pas la modification.

8.2. Procédure

1. Dans Burp → **Proxy** → **Intercept**, cliquer sur le bouton pour activer : **"Intercept is on"**.
2. Dans l'émulateur, rafraîchir une page (F5 ou bouton rafraîchir).
3. Observer dans Burp que la requête est **"en attente"** (elle ne passe pas).
4. Cliquer sur **"Forward"** pour laisser passer la requête.
5. **Immédiatement** désactiver l'interception : **"Intercept is off"**.

Commande(s)

```
# Via ADB     observer l' tat   r seau pendant l'interception :
adb shell netstat -an | grep ESTABLISHED

# En mode passif (history uniquement), le trafic passe librement
    .
# En mode actif (intercept ON), le trafic est bloqu  jusqu'au "
    Forward".

# Si des requ tes s'accumulent, les vider avec "Drop" ou
# d sactiver l'interception directement.
```

[i] Pourquoi cette étape ?

Passif (HTTP history) : les requêtes transitent et sont enregistrées automatiquement — idéal pour l'observation.
Actif (Intercept ON) : chaque requête est bloquée et doit être manuellement validée — utile pour modifier, mais risqué en labo si oublié.

[v] À observer / Résultat attendu

La requête apparaît "en attente" dans l'onglet Intercept.

Après "Forward", la page se charge dans l'émulateur.

L'interception est désactivée immédiatement après l'observation.

[x] Erreurs fréquentes

Laisser l'intercept activé → bloque tout le trafic, lab inutilisable.

Tenter de modifier les requêtes (hors objectif de ce lab).

Ne pas cliquer sur "Forward" → l'émulateur attend indéfiniment.

9. Étape 8 — Gestion du certificat CA pour HTTPS

9.1. Description

HTTPS chiffre le trafic entre le navigateur et le serveur. Pour que Burp puisse l'analyser en laboratoire, un certificat CA de test doit être installé dans l'émulateur Android.

9.2. Export du certificat Burp

Commande(s)

```
# M thode 1      Via le navigateur de l'  mulateur   :
# Naviguer vers http://burp (ou http://<IP_HOTE>:8080)
# Cliquer sur "CA Certificate" pour t l charger cacert.der

# M thode 2      Export depuis Burp (menu) :
# Proxy > Proxy settings > Import/Export CA Certificate
# Choisir "Export Certificate in DER format"
# Enregistrer sous : /home/user/burp_ca.der

# M thode 3      Via ADB, pousser le certificat dans l'
    mulateur   :
adb push /home/user/burp_ca.der /sdcard/Download/burp_ca.der
```

9.3. Installation dans Android

Commande(s)

```
# Depuis l'  mulateur   Android (interface graphique) :
# Param tres > S curit   > Installer depuis le stockage
# S lectionner burp_ca.der
# Choisir : "CA Certificate"
# Nommer : "BurpLab"
# Confirmer le PIN si demand .

# V rification ADB que le certificat est install  :
adb shell ls /data/misc/user/0/cacerts-added/

# Alternative pour Android 7+ (user cert) :
# Le certificat peut n cessiter une configuration Network
   Security Config
# dans l'application cible pour  tre  reconnu.
```

Type de certificat	Usage
CA Certificate	Autorité de confiance — nécessaire pour HTTPS
VPN & App user cert	Pour VPN / applications spécifiques
Wi-Fi certificate	Pour authentification WPA Enterprise

[!] Point de vigilance

Ce certificat CA réduit la sécurité de l'émulateur. Il est **strictement limité** à l'environnement de laboratoire. Ne jamais installer un certificat Burp sur un appareil personnel ou de production. Retirer le certificat en fin de séance.

[x] Erreurs fréquentes

Installer le certificat sur un téléphone réel personnel.

Oublier de retirer le certificat à la fin du lab.

Tester HTTPS sans avoir installé le certificat → erreur SSL dans le navigateur.

10. Étape 9 — Production du mini-rapport

10.1. Description

Un audit sans documentation n'a aucune valeur. Cette étape consiste à produire une fiche de preuve reproductible.

10.2. Structure du rapport

Commande(s)

```
# Cr er le fichier rapport depuis le terminal Mobexler :
mkdir -p ~/lab_reports/$(date +%Y%m%d)
cd ~/lab_reports/$(date +%Y%m%d)

# Informations    collecter automatiquement :
echo "=== RAPPORT DE LAB ===" > rapport.txt
echo "Auteur : BOUSEDRA Ilyas" >> rapport.txt
echo "Date   : $(date)" >> rapport.txt
echo "IP Hote: $(hostname -I | awk '{print $1}')" >> rapport.txt
echo "Port   : 8080" >> rapport.txt
burpsuite --version 2>/dev/null >> rapport.txt
echo "=== FIN CONFIGURATION ===" >> rapport.txt
```

10.3. Contenu requis du rapport

Section	Contenu
Périmètre	"Émulateur de labo, cible autorisée"
Configuration	Version Burp, `<IP_HOTE>`, `<PORT>`, date/heure
Preuves	Capture HTTP history (1-2 requêtes), détail headers + URL
Observations	Données qui transitent, cookies, tokens, paramètres
Risques	Tokens en URL, absence de headers sécurité, cookies non sécurisés
Recommandations	Minimiser données, sécuriser cookies serveur, Network Security Config Android

[i] Pourquoi cette étape ?

La qualité d'un audit se mesure à la **qualité de la documentation**. Un rapport reproductible permet à n'importe quelle autre personne de refaire exactement le même test dans le même environnement.

[!] Point de vigilance

Un bon rapport distingue clairement trois niveaux : **ce qui est observé** (faits), **ce qui est supposé** (hypothèses), et **ce qui est recommandé** (actions correctives).

11. Nettoyage — Hygiène de fin de séance

11.1. Description

Un laboratoire doit être remis dans son état initial après chaque séance pour ne pas "contaminer" les sessions suivantes.

Commande(s)

```
# 1. Retirer le proxy dans l'  mulateur   via ADB :
adb shell settings delete global http_proxy
adb shell settings delete global global_http_proxy_host
adb shell settings delete global global_http_proxy_port

# V rification :
adb shell settings get global http_proxy
# R sultat attendu : null

# 2. Retirer le certificat CA depuis Android :
# Param tres > S curit  > Certificats de confiance >
   Utilisateur
# Appuyer sur "BurpLab" > Supprimer

# Via ADB (si acc s root sur l'  mulateur  ) :
adb shell rm /data/misc/user/0/cacerts-added/<hash>.0

# 3. Dans Burp Suite :
# Fermer le projet temporaire (ne pas sauvegarder)
# File > Close project

# 4. Archiver uniquement les captures n cessaires :
tar -czf ~/lab_reports/archive_$(date +%Y%m%d).tar.gz \
    ~/lab_reports/$(date +%Y%m%d)/

# 5. V rification finale de l' tat  de l'  mulateur  :
adb shell settings get global http_proxy
# Doit retourner : null
```

[!] Point de vigilance

Oublier de retirer le certificat CA laisse l'émulateur **vulnérable** à toute future interception. Oublier de désactiver le proxy rend l'émulateur inutilisable lors de la prochaine session si Burp n'est pas lancé.

12. Checkpoints de validation

N°	Checkpoint	Validé
1	Burp capture au moins une requête dans HTTP history	
2	Proxy listener actif sur port 8080, All interfaces	
3	Proxy Android en “Manual” avec `<IP_HOTE>:8080`	
4	Intercept utilisé brièvement pour démonstration puis désactivé	
5	Au moins une requête analysée (méthode, headers, cookies)	
6	Mini-rapport produit (preuve + contexte)	
7	Proxy Android remis sur “Aucun” en fin de séance	
8	Certificat CA retiré de l’émulateur	
9	Projet Burp fermé sans sauvegarde	

13. Conclusion

Ce laboratoire a permis de mettre en place une chaîne complète d’interception de trafic mobile dans un environnement contrôlé avec Mobexler. Les compétences acquises sont :

]Configuration d’un proxy Burp Suite accessible depuis un émulateur Android. Identification et documentation des paramètres réseau de labo. Lecture et analyse d’une requête HTTP : méthode, headers, cookies, paramètres. Distinction entre mode passif (observation) et mode actif (interception). Gestion du modèle de confiance HTTPS en labo (certificat CA temporaire). Production d’un rapport documenté, reproductible et structuré. Hygiène de fin de séance : retour à un état sain.

Rappel éthique et légal

Ce laboratoire est réalisé dans un cadre **strictement pédagogique**, sur des cibles autorisées, dans un environnement isolé. L’utilisation de ces techniques sur des systèmes sans autorisation explicite est **illégale** et expose à des poursuites pénales.

[v *Rapport rédigé par* ***BOUSEDRA Ilyas***
February 21, 2026